

How many times does your computer just sit there doing nothing after you entered a command. There is no disk activity; no screen activity; nothing. But, you can't do anything since the application or the operating system is "locked up". This is because the software isn't handling the network (flakiness) correctly. The network will continue to be very error prone (e.g., different delays, outages, errors, node failures, etc.) for quite some time in the future. Moreover, the fact that we want to be mobile moving between wireless cells or connecting between very different speed networks means that handling networking resiliency must be a top priority for us. Architecturally, we need to learn about network errors in a standard way and then avoid multiple timeouts, nested retries, etc.

We do have parts of the operating system that are more resilient today than other parts. One example is scandisk that automatically "repairs" the system if there are disk inconsistencies detected. The issue today is that scandisk may not correct the problem totally. In fact, it could delete a critical file. Still it is better than not doing anything. Assuming that we must have a utility like scandisk (because of the file system design) we need to ensure that upper levels of the software can recover from perhaps poor deletion choices that scandisk might make. (Furthermore, as noted later, if we have to have scandisk utilities, they should run silently without introducing the user to any bizarre terminology or operational choices.)

Resiliency is about keeping the system running at all costs. An example of this is that if the NT 4 shell (or IE 4 shell on NT 5) dies, the desktop disappears for a second. But, the operating system restarts the shell quietly in the background. The system is resilient to shell faults. This is not the case with win95 or win98.

Ways to improve resiliency:

- Auto-configure. Do not let the user configure things. Networking is one of the most complicated areas of our system. Asking a novice (or even an "expert") to configure a protocol stack is like asking a layperson to do surgery – it takes training and even then they sometimes make a mistake. If you can't auto-configure, then the system should probably be redesigned so that it can be auto-configured.
- Assume a machine is transiently connected to the network. Never allow the user interface to be stopped by a network outage. Switch gracefully (but visibly) from network mode to offline mode.
- Once an error is detected (e.g., network timeout), it should be sensed only once. That is, at most one timeout should happen until there is good communications established again.
- Design for error cases even if it means the regular operation of the system is impacted slightly. A good example here is out of disk space. Assume you run out of disk space at the worst possible time. Will the system still function correctly? Can the user still remove some space? If it takes allocating some free disk space as a buffer during normal operation (to ensure that the system is still totally functional) then do it.

Terminology

For some strange reason we insist on trying to teach people a foreign language: *geektalk*. My Mom doesn't want to learn another language right now. She just wants to track her church work, write letters, exchange email, track her budget, look for things on the Internet, etc.

Geektalk is actually one of our worse sins in interacting with a user. Our systems tend to produce more internal information than a user needs to know and in doing so we use terms that only a computer literate person (and in some cases programmer) would understand.

Turn on (note a naïve user thinks a "boot" is a shoe) any PC and just watch the words/characters fly by.